

# Practice Guide 3 (advanced): Breaking the Spam Filter

Aprendizagem Aplicada à Segurança

Mário Antunes

October 9, 2026

## Objective

You play the **attacker** against the spam filters built in Guides 1 and 2. **Lab use only:** you attack the models you trained yourself. Before writing code, state the **threat model**.

|                   |                                                                                                        |
|-------------------|--------------------------------------------------------------------------------------------------------|
| Axis              | This lab                                                                                               |
| <b>Goal</b>       | a spam message is classified as ham ( <i>evasion</i> ); or the model is corrupted ( <i>poisoning</i> ) |
| <b>Capability</b> | append words to a message; submit “not spam” feedback                                                  |
| <b>Knowledge</b>  | black-box: a score per message (each score is a <i>query</i> )                                         |
| <b>Constraint</b> | the phone number, link and call to action stay intact                                                  |

- **Notebook to complete:** `guide_03.ipynb`. **Time:** about 2 hours. Optional, advanced. **Prerequisite:** Guides 1 and 2.

## Background

**Derivative-free search.** The attacker cannot differentiate the filter; they *evaluate* it. Greedy search: at each step try appending every candidate word, keep the one that lowers the spam score the most, stop when the score becomes negative (the message goes to the inbox) or the budget runs out.

**Cost.** With  $m$  candidates and  $s$  steps the attack uses about  $1 + m \cdot s$  queries. Rate limiting, returning only labels, and detecting bursts of near-identical messages all raise this cost.

**Poisoning.** A filter that retrains on user feedback can be taught a **backdoor**: spam

carrying a rare trigger token, reported as *not spam*, makes the model learn that the trigger means ham. The clean test score barely moves.

## Tasks

| Part     | What you do                                                      | Check                        |
|----------|------------------------------------------------------------------|------------------------------|
| <b>A</b> | Candidate words: the 40 most frequent ham words (given)          |                              |
| <b>B</b> | Implement <code>greedy_append</code> (black-box greedy search)   | the <code>assert</code> cell |
| <b>C</b> | Evasion rate versus budget (0 to 15 words), queries per message  | plot                         |
| <b>D</b> | Poisoning with a trigger token: 0, 10, 50, 100 poisoned messages | table                        |

## Questions for the report

1. **(C1)** Plot the evasion curves of Naive Bayes and logistic regression. Which one is easier to break, and why? How many queries per message did each attack need?
2. **(C2)** At budget 0 some messages already evade. What are they, and how does this change the meaning of “attack success rate”?
3. **(C3)** The clean F1 of both filters is about 0.95. What does that tell you about accuracy as a security metric?
4. **(C4)** Suggest two ways to raise the attacker’s cost (per query and per message) and one way the attacker could adapt to each.
5. **(D1)** Why does the clean spam-caught rate stay at 0.91 while the triggered rate collapses? How would you detect a backdoor like this?
6. **(D2)** Repeat part D with Naive Bayes. Is it as vulnerable? Explain with the way each model turns a word into evidence.

## Optional challenges

- Limit the appended words to at most 5 *and* penalise rare words: how does the success rate change?
- Attack the **fastText** model of Guide 2 with the same greedy search.
- **Adversarial training:** append the words found by the attack to the training spam (label *spam*), retrain, and re-run part C. What does the attacker do next?

## Grading

Working implementations that pass the checks (50%), the plots and tables (25%), the answers to the questions (25%). Reference solution: `solution/guide_03.ipynb`.